

Звіт про оригінальність

● Оцінка схожості

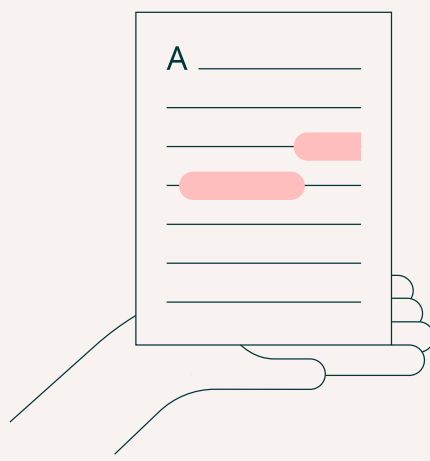
% 4

● Ризик плагіату

СЕРЕДНІЙ

👤 Offiks 🕒 2026-09-16 14:29

Посилання на звіт: 1dlk2 / Посилання користувача: sjWr



Ось вона – Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- ☒ База даних інтернет-джерел
- ☐ База даних наукових статей
- ☐ Глибока перевірка (наш вдосконалений алгоритм)

Бали



● Збіги тексту	4%
● Перефразування	0%
● Цитований текст	0%
● Неправильне цитування	0%
● Збігів не знайдено	96%

Ризик плагіату

СЕРЕДНІЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.

%

4

Збіги

5 МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

1 ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА

3 Факультет математики, фізики та інформаційних технологій

3 Кафедра математичного забезпечення комп'ютерних систем

3 КУРСОВИЙ ПРОЄКТ

з освітнього компоненту «Програмування»

на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ

«Спрощений варіант №19»

Студента 1 курсу, 10 денна форма навчання спеціальність F7 «Комп'ютерна інженерія»

Цуркана Максима Сергійовича

Керівник: к.ф.- м.н., доц. Антоненко О.С.

5 Національна шкала: _____

5 Кількість балів: _____ ECTS: _____

Дата захисту: _____

Члени комісії ____ Антоненко О.С.

1 (підпис) (прізвище та ініціали)

1 ____ 1 Шестопапов С.В.

1 (підпис) (прізвище та ініціали)

1 ____ 1 _____

1 (підпис) (прізвище та ініціали)

1 ОДЕСА – 2026

ЗМІСТ

стор.

ПОСТАНОВКА ЗАВДАННЯ3

1.ТЕОРЕТИЧНІ ВІДОМОСТІ4

2.СТРУКТУРА ПРОГРАМИ6

2.1 Ієрархія класів та їх функціонал6

2.2Допоміжні функції та користувацький інтерфейс9

3.ІНТЕРФЕЙС12

ВИСНОВКИ18

СПИСОК ЛІТЕРАТУРИ19

ДОДАТОК А. Діаграма класів20

ДОДАТОК Б. Код основних класів21

ПОСТАНОВКА ЗАВДАННЯ

Створити клас `Person`, який містить такі поля: ПІБ (`fullName`), зріст (`height`), вагу (`weight`), стать (`gender`), країну проживання (`country`) та електронну пошту (`email`). У класі необхідно передбачити конструктор без параметрів для ініціалізації за замовчуванням та конструктор з параметрами, в якому виконується обов'язкова перевірка коректності вхідних даних (валідація зросту та ваги). Реалізувати спеціалізований метод для зміни електронної пошти, а також окремі логичні методи для перевірки умов: чи перевищує зріст людини 200 см та чи є країною проживання Україна. Для забезпечення віртуалізації виведення реалізувати метод `print()`, `to_string()` або перевантажити оператор вставки у потік `operator<<`.

Реалізувати два класи-спадкоємці — `Worker` (Робітник) та `Electrician` (Електрик), які розширюють базовий клас `Person` власними специфічними атрибутами (наприклад, стаж, підприємство або група допуску). У кожному з похідних класів обов'язково перевизначити метод виведення інформації (`print()`, `to_string()` або `operator<<`) для коректного відображення як базових полів, так і нових даних.

В основній функції main() реалізувати гнучкий сценарій взаємодії через інтерактивне текстове консольне меню. Користувач повинен мати можливість виконувати будь-які дії (додавання, видалення, пошук, виведення) у довільному порядку без примусової послідовності. Для запобігання витокам пам'яті забезпечити наявність віртуального деструктора в базовому класі Person та коректне звільнення динамічно виділеної пам'яті під час видалення елементів або завершення роботи програми.

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

2 Об'єктно-орієнтоване програмування (ООП) — парадигма програмування, в основі якої лежить концепція представлення програми у вигляді сукупності об'єктів. Кожен об'єкт є екземпляром певного класу, що об'єднує в собі як дані (стан), так і методи для їхньої обробки (поведінку) [1]. Такий підхід дозволяє моделювати реальні або абстрактні сутності, спрощуючи структурування та підтримку складних програмних систем.

Клас є користувацьким типом даних, який слугує шаблоном або «кресленням»

2 для створення об'єктів. Він визначає набір атрибутів (полів) та операцій (методи). Об'єкт — це конкретний екземпляр класу, що існує в пам'яті під час виконання програми та має власні значення полів.

Інкапсуляція — принцип ООП, який полягає в об'єднанні 2 даних і методів їхньої обробки всередині однієї сутності (класу) та приховуванні внутрішніх деталей реалізації від зовнішнього світу. Взаємодія з об'єктом здійснюється лише через чітко визначений публічний інтерфейс, що захищає внутрішній стан від некоректного прямого втручання та підтримує цілісність даних [2].

Успадкування — механізм, що дозволяє створювати нові класи на основі вже існуючих. Похідний клас (клас-спадкоємець) повторно використовує елементи базового класу, за потреби доповнюючи їх новими полями та методами або змінюючи наявну поведінку. Це забезпечує ієрархічну побудову програмних продуктів та мінімізує дублювання коду[3, 2].

Поліморфізм — властивість, яка дозволяє використовувати єдиний інтерфейс для об'єктів різних типів, що належать до однієї ієрархії. Завдяки поліморфізму програма може викликати однакові за назвою методи, але під час виконання буде виконана реалізація, відповідна конкретному типу об'єкта [4, 5]. Забезпечується це через використання віртуальних методів та механізм динамічного (пізнього) зв'язування.

СТРУКТУРА ПРОГРАМИ

Ієрархія класів та їх функціонал

У розробленій програмній системі об'єктно-орієнтована архітектура побудована на основі ієрархії класів, що моделює сутності людей та їхніх професійних ролей. Головним елементом ієрархії є базовий клас `Person`, від якого успадковуються класи `Worker` (Робітник) та `Electrician` (Електрик). Також у системі присутній допоміжний клас `Object`, який описує виробничий об'єкт і використовується для перевірки професійних допусків.

Базовий клас «`Person`» є основою ієрархії та відповідає за зберігання й обробку загальної інформації про людину;

Специфікатор доступу даних: Для забезпечення доступу до полів базового класу у похідних класах використано модифікатор `protected`;

Поля класу:

`fullName (std::string)` — прізвище, ім'я та по батькові;

`height (int)` — зріст у сантиметрах;

`weight (int)` — вага у кілограмах;

`gender (std::string)` — стать;

`country (std::string)` — країна проживання;

`email (std::string)` — адреса електронної пошти.

Методи класу:

Конструктори: Реалізовано конструктор за замовчуванням (ініціалізує поля базовими значеннями) та конструктор з параметрами. У конструкторі з параметрами передбачено механізм валідації вхідних даних: у разі спроби передачі від'ємних значень зросту чи ваги генерується виключна ситуація `std::invalid_argument`;

Модифікатор даних `changeEmail(const string& e)`: Дозволяє безпечно оновити адресу електронної пошти;

`height_above_200()` — повертає `true`, якщо зріст людини перевищує 200 см;

`live_in_Ukraine()` — повертає `true`, якщо країною проживання вказано «Україна» або «Ukraine»;

Віртуальний метод `print()`: Призначений для виведення форматованої інформації про об'єкт у консоль. Оголошення методу як `virtual` забезпечує можливість його перевизначення у похідних класах для реалізації динамічного поліморфізму;

Метод `printName()`: Виводить у консоль лише ПІБ людини, що зручно для використання у списках вибору;

Віртуальний деструктор `virtual ~Person() = default`: Забезпечує коректне звільнення пам'яті об'єктів похідних класів під час їх видалення через вказівник на базовий клас.

Похідний клас «Worker» (Робітник) публічно успадковує клас `Person` (`public Person`), розширюючи його атрибутами та специфікою трудової діяльності робітника на підприємстві;

Власні поля класу:

`factory (std::string)` — назва заводу/підприємства;

`position (std::string)` — займана посада;

`salary (int)` — розмір посадового окладу.

Методи класу:

Конструктор: Приймає параметри базового класу та власні атрибути робітника, передаючи загальні дані до конструктора `Person` через список ініціалізації;

`increaseSalary()`: Власний метод класу, який здійснює автоматичне підвищення окладу робітника на фіксовану суму залежно від його посади (наприклад, для посади «Assembler» додається 1000 грн, для «Director» — 3000 грн);

`changePos(const string& pos)`: Метод для зміни поточної посади робітника;

Перевизначений метод `print() const override`: Повністю заміщує базову реалізацію, виводячи як загальні персональні дані, так і інформацію про завод, посаду та оклад робітника.

Допоміжний клас «Object» (Виробничий об'єкт) не належить до ієрархії «Person», а виступає як самостійна сутність, що описує робочий об'єкт (наприклад, промисловий майданчик чи дільницю);

Поля класу:

`name (std::string)` — назва об'єкта;

`reqExperience (int)` — мінімальний стаж роботи, необхідний для допуску;

`reqRank (int)` — мінімальний професійний розряд, необхідний для допуску.

Методи класу: Конструктор з параметрами та методи-гетери (returnName(), returnReqExperience(), returnReqRank()) для отримання значень приватних полів.

Похідний клас «Electrician» (Електрик) також є прямим спадкоємцем «Person» (public Person). Він моделює фахівця з електромонтажу, який прив'язаний до певного виробничого об'єкта.

Власні поля класу:

experience (int) — стаж роботи електрика (у роках);

rank (int) — професійний розряд;

currentObject (Object*) — вказівник на об'єкт типу Object, на якому наразі працює або до якого прикріплений електрик.

Методи класу:

Конструктор: Ініціалізує базові поля та специфічні дані електрика, включаючи асоціацію з об'єктом через вказівник;

meetRequirements() (без параметрів) — перевіряє, чи відповідають стаж та розряд електрика вимогам його поточного об'єкта (currentObject);

meetRequirements(Object* object) (з параметром) — перевіряє, чи може електрик працювати на будь-якому іншому довільному об'єкті, переданому через вказівник;

changeObject(Object* newObject): Змінює поточний робочий об'єкт електрика;

Перевизначений метод print() const override: Виводить повну інформацію про людину, включаючи стаж, розряд та назву об'єкта, на якому вона працює.

.

Допоміжні функції та користувацький інтерфейс

Для забезпечення взаємодії користувача з консольною системою та виведення списків у програмі реалізовано окремі сервісні функції. Вони відповідають за формування меню, форматування списків елементів для вибору та захист від некоректного введення даних.

Модулі формування текстових меню:

Функція gui(): Відповідає за виведення в консоль переліку основних команд головного меню програми (показує 8 основних опцій управління об'єктами та команду

завершення роботи 0);

Функція `addPerson()`: Формує текстове підменю вибору типу об'єкта (`Person`, `Worker` або `Electrician`) під час виконання процедури створення нового запису;

Функції навігації та вибору елементів зі списків;

Функція `choicePerson(const vector<Person*>& vec)`: Приймає посилання на вектор вказівників базового класу `Person`. Здійснює обхід масиву та виводить у консоль пронумерований список персоналій із викликом методу `printName()` для кожного елемента. Використовується для надання користувачеві можливості вибору конкретної особи за її порядковим номером у процедурах виведення, видалення, зміни email та виконання професійних дій;

Функція `choiceObject(const vector<Object*>& vec)`: Приймає посилання на вектор вказівників `Object` та формує пронумерований список назв наявних виробничих об'єктів. Застосовується при прив'язці електрика до об'єкта або при виконанні дій перевірки допусків.

Обробка некоректного введення даних та перевірка цілісності

Контроль вхідного потоку (`cin.fail()`): При зчитуванні числових пунктів меню та атрибутів (зріст, вага, оклад, стаж, розряд) використовується перевірка прапорця помилки потоку. У разі введення символьних даних замість чисел виконується скидання стану потоку (`cin.clear()`) та очищення буфера (`cin.ignore()`), після чого запит повторюється;

Перевірка межових значень: Усі функції та блоки вибору здійснюють перевірку введеного індексу на відповідність діапазону елементів у векторах (від 1 до `vec.size()`), що унеможлиблює виникнення помилок виходу за межі масиву (`Out of Range`).

ІНТЕРФЕЙС

При запуску програми користувач потрапляє до головного меню, в якому представлено основні можливості програми (Рис. 3.1). За допомогою введення номера користувач може обрати необхідну дію: переглянути інформацію про людину, додати або видалити людину, додати об'єкт, змінити електронну пошту, виконати дії з робітником або електриком, а також переглянути електриків, які можуть працювати на вибраному об'єкті.

Рисунок 3.1 – Початковий інтерфейс

Якщо користувач обирає перший пункт головного меню, **9** програма виводить список усіх наявних людей та пропонує обрати необхідну людину (Рис. 3.2). Після вибору

виводиться повна інформація про неї.

Рисунок 3.2 – Всі доступні люди

Після вибору необхідної людини зі списку програма виводить повну інформацію про неї. На екрані відображаються ПІБ, зріст, вага, стать, країна проживання та електронна пошта (Рис. 3.3).

Рисунок 3.3 – Виведення інформації про людину

При виборі другого пункту користувач може додати новий об'єкт одного з трьох типів: звичайну людину, робітника або електрика (Рис. 3.4).

Рисунок 3.4 – Вибір типу людини для додавання

Після вибору типу людини користувач послідовно вводить необхідні дані: ПІБ, зріст, вагу, стать, країну та електронну пошту. Після коректного введення всіх значень створюється новий об'єкт відповідного класу та додається до загального списку людей (Рис. 3.5).

Рисунок 3.5 – Додавання нової людини

Під час введення номера пункту меню програма перевіряє коректність введених даних. Якщо користувач замість числа вводить інший символ, програма повідомляє про помилку та повторно виводить головне меню (Рис. 3.6).

Рисунок 3.6 – Перевірка введення

При виборі третього пункту програма виводить список людей. Користувач обирає людину, яку необхідно видалити, після чого відповідний об'єкт видаляється з пам'яті та списку (Рис. 3.7).

Рисунок 3.7 – Видалення людини

Після вибору необхідної людини зі списку програма видаляє відповідний об'єкт із пам'яті та прибирає його із загального списку людей. Після успішного виконання операції користувач отримує відповідне повідомлення.

Рисунок 3.8 – Успішне видалення людини

Четвертий пункт головного меню використовується для створення нового об'єкта. Для об'єкта користувач задає його назву, мінімальний необхідний стаж та розряд електрика (Рис. 3.9).

Рисунок 3.9 – Додавання об'єкта

У п'ятому пункті користувач може обрати будь-яку людину зі списку та змінити її електронну адресу (Рис. 3.10).

Рисунок 3.10 – Зміна електронної пошти

Шостий пункт дозволяє виконувати спеціальні дії з об'єктом класу Worker. Після вибору людини програма перевіряє, чи є вона робітником. Для робітника доступне збільшення окладу та зміна посади (Рис. 3.11).

Рисунок 3.11 – Дії з робітником

Після вибору робітника програма надає користувачу можливість виконати одну з доступних дій: збільшити оклад або змінити посаду. Після вибору необхідного пункту програма виконує відповідну операцію для обраного робітника (Рис 3.12).

Рисунок 3.12 – Можливості дій з робітником

Якщо користувач обирає людину, яка не належить до класу Worker, програма визначає це та виводить повідомлення про помилку. Після цього виконання дій, доступних лише для робітника, не відбувається (Рис 3.13).

Рисунок 3.13 – Помилка при виборі людини, яка не є робітником

Сьомий пункт призначений для роботи з електриками. Користувач може перевірити допуск електрика до поточного об'єкта або призначити йому інший об'єкт (Рис. 3.14).

Рисунок 3.14 – Дії з електриком

Після вибору електрика програма перевіряє, чи відповідають його стаж та розряд вимогам поточного об'єкта. Якщо обидві умови виконуються, програма виводить повідомлення про те, що електрик допускається до роботи на об'єкті (Рис 3.15).

Рисунок 3.15 – Успішна перевірка допуску електрика до роботи на об'єкті

Якщо стаж або розряд вибраного електрика не відповідає встановленим вимогам об'єкта, програма забороняє допуск до роботи та виводить відповідне повідомлення. У такому випадку електрик не може виконувати роботи на вибраному об'єкті (Рис 3.16).

Рисунок 3.16 – Відмова у допуску електрика до роботи на об'єкті

Після вибору відповідного пункту користувач може змінити об'єкт, на якому працює електрик. Програма виводить список доступних об'єктів, після чого користувач обирає новий об'єкт. У разі коректного вибору поточний об'єкт електрика змінюється, а програма виводить **8** повідомлення про успішне виконання операції (Рис. 3.17).

Рисунок 3.17 – Зміна об'єкта електрика

При виборі восьмого пункту користувач спочатку обирає об'єкт. Після цього програма перевіряє всіх людей, знаходить серед них об'єкти класу Electrician та перевіряє відповідність їхнього стажу і розряду вимогам вибраного об'єкта. У результаті виводиться список електриків, які можуть виконувати роботи на ньому (Рис. 3.18).

Рисунок 3.18 – Пошук електриків для об'єкта

При виборі восьмого пункту користувач спочатку обирає об'єкт. Після цього програма перевіряє всіх людей, знаходить серед них об'єкти класу Electrician та перевіряє відповідність їхнього стажу і розряду вимогам вибраного об'єкта. У результаті виводиться список електриків, які можуть виконувати роботи на ньому (Рис. 3.19).

Рисунок 3.19 – Пошук електриків для об'єкта

ВИСНОВКИ

У результаті виконання курсового проєкту було розроблено консольну програму мовою C++, яка реалізує систему роботи з людьми, робітниками, електриками та об'єктами. У програмі створено базовий клас Person, а також похідні класи Worker та Electrician, що дозволило реалізувати наслідування та поліморфізм. Для виведення інформації про об'єкти використовується перевизначення віртуального методу print().

Під час розробки було реалізовано основні можливості роботи з об'єктами: додавання та видалення людей, створення нових об'єктів, зміну електронної пошти, зміну посади та збільшення окладу робітника, а також роботу з електриками. Для електрика передбачена перевірка відповідності його стажу та розряду вимогам певного об'єкта, можливість зміни поточного об'єкта та пошук електриків, які можуть бути допущені до виконання робіт.

Для взаємодії користувача з програмою було створено консольний інтерфейс з головним меню, через яке можна обрати необхідну операцію. Також у програмі реалізовано перевірку коректності введених даних, що дозволяє уникнути виконання операцій з неправильними значеннями.

Таким чином, у ході виконання курсового проєкту було закріплено практичні навички роботи з класами, наслідуванням, поліморфізмом, віртуальними методами, динамічним приведенням типів, векторами та динамічним створенням об'єктів у C++. Розроблена програма виконує поставлені завдання та забезпечує взаємодію між створеними класами через зручний консольний інтерфейс.

СПИСОК ЛІТЕРАТУРИ

Порєв В. М. 4 Об'єктно-орієнтоване програмування. Конспект лекцій / 4 Навчальний посібник. – 4 КПІ ім. Ігоря Сікорського, 2022.

<https://ela.kpi.ua/items/b642efee-5d40-4469-80fe-2b6b5ca89037>

Основні поняття ООП(Інкапсуляція, успадкування, поліморфізм) –

<https://studfile.net/preview/5851327/page:11/>

Репозиторій GitHub на курсовий – Режим доступу: <https://github.com/dolda-cyber/kursov>

ДОДАТОК А. 6 Діаграма класів

6 Рисунок А.1 – Діаграма класів

6 ДОДАТОК Б. Код основних класів

```
class Person {  
  
protected:  
  
string fullName;  
  
int height;  
  
int weight;  
  
string gender;  
  
string country;  
  
string email;  
  
public:  
  
Person()  
  
: fullName("Невідомо"), height(0), weight(0),  
gender("Невідомо"), country("Не вказано"),  
email("Відсутнє") {}  
  
Person(const 7 string& fn, int h, int w, const string& g,  
const string& c, const string& email)  
  
: fullName(fn), height(h), weight(w),
```

```

gender(g), country(c), email(email) {

if (h < 0)

throw invalid_argument("Від'ємний зріст!");

if (w < 0)

throw invalid_argument("Від'ємна вага!!");

}

void changeEmail(const string& e) {

email = e;

}

virtual void print() const {

cout << "\nПІБ: " << fullName

<< "\nЗріст: " << height

<< "\nВага: " << weight

<< "\nСтать: " << gender

<< "\nКраїна: " << country

<< "\nEmail: " << email << "\n";

}

virtual ~Person() = default;

};

```

Лістинг Б.1 – Клас Person

```

class Worker : public Person {

string factory;

string position;

```

```

int salary;

public:
Worker(const 7 string& fn, int h, int w, const string& g,
const string& c, const string& email,
const string& fact, const string& pos, int salary)
: Person(fn, h, w, g, c, email),
factory(fact), position(pos), salary(salary) {}

void increaseSalary() {
if (position == "Assembler")

salary += 1000;

else if (position == "Operator")

salary += 1500;

else if (position == "Machinist")

salary += 1750;

else if (position == "Supervisor")

salary += 2500;

else if (position == "Director")

salary += 3000;

}

void changePos(const string& pos) {

position = pos;

}

void print() const override {

cout << "\nПІБ: " << fullName

```

```

<< "\nЗавод: " << factory
<< "\nПосада: " << position
<< "\nОклад: " << salary << "\n";
}
};

```

Лістинг Б.2 – Клас Worker

```

class Object {
int reqExperience;
int reqRank;
string name;
public:
Object(int reqExp, int reqRank, const string name)
: reqExperience(reqExp),
reqRank(reqRank),
name(name) {}
int returnReqExperience() {
return reqExperience;
}
int returnReqRank() {
return reqRank;
}
string returnName() {
return name;
}
}

```

```
};
```

ЛІСТИНГ Б.3 – Клас Object

```
class Electrician : public Person {  
  
    int experience;  
  
    int rank;  
  
    Object* currentObject;  
  
public:  
  
    Electrician(const string& fn, int h, int w,  
                const string& g, const string& c,  
                const string& email, int exp,  
                int rank, Object* currObj)  
        : Person(fn, h, w, g, c, email),  
          experience(exp), rank(rank),  
          currentObject(currObj) {}  
  
    bool meetRequirements() {  
  
        return experience >= currentObject->returnReqExperience()  
            && rank >= currentObject->returnReqRank();  
    }  
  
    bool meetRequirements(Object* object) {  
  
        return experience >= object->returnReqExperience()  
            && rank >= object->returnReqRank();  
    }  
  
    void changeObject(Object* newObject) {  
  
        currentObject = newObject;  
    }  
};
```

```
}  
  
void print() const override {  
  
    cout << "\nПІБ: " << fullName  
  
    << "\nСтаж: " << experience  
  
    << "\nРозряд: " << rank  
  
    << "\nОб'єкт: " << currentObject->returnName()  
  
    << "\n";  
  
}
```

Лістинг Б.4 – Клас Electrician

Посилання

Це джерела виділених збігів у вашому документі. Кожен збіг позначено темно-зеленим числом, яке відповідає вказаному тут джерелу. Джерела впорядковані за схожістю — чим вищий бал, тим сильніше збіг.

#	Джерело	%
1	dspace.onu.edu.ua	0.8%
2	dspace.mnau.edu.ua	0.7%
3	onu.edu.ua	0.4%
4	ela.kpi.ua	0.4%
5	essuir.sumdu.edu.ua	0.4%
6	iq.vntu.edu.ua	0.3%
7	aclanthology.org	0.3%
8	lpnu.ua	0.3%
9	ela.kpi.ua	0.2%
10	wunu.edu.ua	0.2%



Дякуємо, що перевірили
свій документ за допомогою
Plag!